

TP Base de donnée

Les vues SQL avec la base Kaggle Olist

1. Liste des vues créées

Au cours de ce TP, 15 vues SQL ont été créées sur la base de données Olist. Chaque vue répond à un objectif précis : simplification, sécurité, reporting ou analyse.

Nom de la vue	Tables sources
vue_clients	customers
vue_clients_sp	customers
vues_commandes_paiements	orders,payments
vue_articles_produits	order_items,products
vue_ventes_detaillees	customers,orders,order_items,products
vue_ca_par_categorie	order_items,products
vue_reporting_commandes	customers,orders
vue_clients_public	customers
vue_commandes_livrees	orders
vue_retard_livraison	orders
vue_top_livraison	order_items,products
vue_paiements_resume	payments
vue_performance_vendeurs	order_items
vue_ca_par_catégorie_mat	order_items,product(matérialisée)

2. Rôle de chaque vue

vue_clients : Expose uniquement les colonnes customer_id, customer_city et customer_state. Simplifie les requêtes et masque les données inutiles de la table customers.

vue_clients_sp : Filtre la table customers pour ne retourner que les clients de l'État SP. Évite de répéter la clause WHERE à chaque requête.

vue_commandes_clients : Jointure entre orders et customers. Permet d'obtenir en une seule requête les informations de commande enrichies de la localisation du client.

vue_commandes_paiements : Jointure entre orders et payments. Facilite l'analyse des paiements associés à chaque commande (type, montant, versements).

vue_articles_produits : Jointure entre order_items et products. Permet d'analyser les ventes par catégorie de produit avec prix et frais de livraison.

vue_ventes_detaillees : Vue analytique combinant 4 tables : customers, orders, order_items et products. Donne une vision complète des ventes par État et catégorie.

vue_ca_par_categorie : Vue agrégée pré-calculant le chiffre d'affaires, le nombre d'articles vendus et le prix moyen par catégorie. Prête à l'emploi pour le reporting.

vue_reporting_commandes : Vue de reporting donnant le nombre de commandes par État et par statut. Idéale pour un tableau de bord commercial.

vue_clients_public : Vue de sécurité exposant uniquement city, state et nombre de clients. Permet de donner un accès restreint sans exposer les données personnelles.

vue_commandes_livrees : Filtre les commandes avec le statut 'delivered'. Simplifie toutes les analyses portant exclusivement sur les commandes livrées.

vue_retard_livraison : Identifie les commandes livrées après la date estimée et calcule le nombre de jours de retard. Utile pour le suivi qualité.

vue_top_categories : Agrège chiffre d'affaires, nombre d'articles vendus et prix moyen par catégorie, triés par CA décroissant.

vue_paiements_resume : Résumé pour chaque type de paiement : nombre de transactions, montant total et montant moyen.

vue_performance_vendeurs : Vue métier analysant les vendeurs : CA généré, nombre de commandes, prix moyen et total livraisons. Aide à identifier les meilleurs vendeurs.

vue_ca_par_categorie_mat : Vue matérialisée (PostgreSQL) stockant physiquement le CA par catégorie. Plus rapide à interroger que la vue classique sur de grands volumes.

3. Comparaison : requête directe vs requête via vue

L'un des objectifs du TP était d'évaluer si l'utilisation d'une vue modifie les performances d'exécution par rapport à une requête directe sur les tables sources.

Mesure des temps d'exécution

La comparaison a été effectuée sur le calcul du chiffre d'affaires par catégorie :

	Requête directe	Requête via vue_articles_produits
Temps d'exécution	2,61 seconde	2,54 secondes
Différence	—	0,07 seconde
Résultats	74 lignes	74 lignes

Analyse

La différence de 0,07 seconde est négligeable. Une vue classique (non matérialisée) ne stocke pas les données physiquement : elle réexécute la requête SQL sous-jacente à chaque appel. Les performances sont donc quasi-identiques à la requête directe.

Vue matérialisée vs vue classique

Critère	Vue classique	Vue matérialisée / Table de synthèse
Stockage	Aucun (virtuelle)	Physique (sur disque)
Mise à jour	Automatique (temps réel)	Manuelle (REFRESH)
Performance	Similaire à la requête directe	Nettement plus rapide (données pré-calculées)
Risque	Lenteur sur grandes tables	Données obsolètes si non rafraîchies

4. Conclusion : intérêt et limites des vues SQL

Intérêts des vues SQL

- **Simplification du code** : une vue remplace une requête complexe (jointures, filtres, agrégations) par un simple nom appellable avec `SELECT * FROM vue`. Le code est plus court, plus lisible et plus facile à maintenir.
- **Réutilisabilité** : une jointure ou une agrégation définie une seule fois dans une vue peut être réutilisée dans de nombreuses requêtes ou applications sans réécriture.
- **Sécurité des données** : en n'exposant que certaines colonnes (ex. `vue_clients_public`), on peut donner à des utilisateurs un accès restreint aux données sans leur montrer les informations sensibles (emails, identifiants, etc.).
- **Facilité pour les non-techniciens** : un analyste ou un responsable commercial peut interroger une vue sans connaître la structure des tables sous-jacentes ni écrire de jointures complexes.
- **Reporting et tableaux de bord** : des vues comme `vue_reporting_commandes` ou `vue_ca_par_categorie` peuvent être connectées directement à des outils BI (Power BI, Tableau) pour afficher des graphiques sans requête SQL supplémentaire

Limites des vues SQL

- **Pas d'amélioration des performances** : une vue classique réexécute sa requête à chaque appel. Sur de grandes tables avec des jointures multiples et des agrégations, cela peut être lent.
- **Pas de mise à jour directe** : une vue contenant des jointures, des agrégations ou des sous-requêtes ne peut généralement pas être mise à jour (`INSERT`, `UPDATE`, `DELETE`) directement.

- **Risque de vues imbriquées** : des vues qui s'appuient sur d'autres vues rendent le code difficile à déboguer et peuvent dégrader les performances de façon exponentielle.
- **Sécurité incomplète** : si un utilisateur a un accès direct à la table source en plus de la vue, les restrictions de la vue peuvent être contournées. Les vues doivent être combinées avec une gestion rigoureuse des droits d'accès.
- **Vue matérialisée** : données potentiellement obsolètes : si l'on utilise une vue matérialisée ou une table de synthèse pour gagner en performance, il faut penser à la rafraîchir régulièrement quand les données sources changent.

Recommandations

⇒ Il convient de créer une vue uniquement lorsqu'elle apporte un réel avantage en termes de simplicité, sécurité ou réutilisabilité. Si une requête n'est utilisée qu'une seule fois, créer une vue alourdit inutilement la base. Pour les tables volumineuses avec des agrégations fréquentes, préférer une vue matérialisée (PostgreSQL) ou une table de synthèse (MySQL) en mettant en place un mécanisme de rafraîchissement régulier. Enfin, les index sur les colonnes de jointure (product_id, order_id, customer_id) restent le levier le plus efficace pour accélérer les requêtes, qu'elles passent par une vue ou non.

Partie 1 - Comprendre le rôle d'une vue

Question 1.1 - Expliquez avec vos propres mots ce qu'est une vue SQL

⇒ Une vue SQL est une table virtuelle qui enregistre une requête SELECT. Elle peut être appelée à tout moment et retourne toujours les données les plus récentes de la table source.

Question 1.2 - Une vue stocke-t-elle physiquement les données comme une table classique ? Justifiez brièvement.

⇒ Non, une vue ne stocke pas physiquement les données contrairement à une table classique. Elle enregistre uniquement la requête SELECT associée et l'exécute à chaque appel, retournant ainsi les données les plus récentes de la table source.

Question 1.4 - Complétez un tableau comparatif entre une table et une vue : stockage physique, interrogation avec SELECT, possibilité de contenir une jointure, intérêt pour simplifier les requêtes.

⇒

	Table	Vue
Stockage physique	Oui	Non
Interrogation avec SELECT	Oui	Oui
Possibilité de contenir une jointure	Non	Oui

Intérêt pour simplifier les requêtes	Non	Oui
--------------------------------------	-----	-----

Question 1.5 - Dans un projet réel, dans quels cas une vue peut-elle être utile pour un développeur ou un analyste ?

⇒ Une vue est utile pour un développeur ou un analyste dans plusieurs cas concrets :

- Un **analyste** peut utiliser une vue pour consulter rapidement des statistiques de ventes mensuelles sans réécrire une requête complexe à chaque fois.
- Un **développeur** peut utiliser une vue pour cacher des données sensibles comme les salaires ou les mots de passe, en n'exposant que les colonnes nécessaires.
- Dans une **application**, une vue contenant une jointure complexe entre plusieurs tables peut être réutilisée à plusieurs endroits du code, ce qui simplifie et accélère le développement.

Partie 2 - Création d' une vue simple

Question 2.1 - Affichez les dix premières lignes de la table customers. Quelles informations trouve-t-on dans cette table ?

SELECT * FROM customers LIMIT 10;

⇒ Les informations qu'on trouve : customer_id, customer_unique_id, customer_zip_code_prefix, customer_city, customer_state.

Question 2.2 - Créez une vue appelée vue_clients contenant uniquement customer_id, customer_city et customer_state.

⇒ CREATE VIEW vue_clients AS
SELECT customer_id, customer_city, customer_state
FROM customers;

Question 2.3 - Interrogez la vue vue_clients. La vue s' utilise-t-elle comme une table dans une requête SELECT ?

SELECT * FROM vue_clients LIMIT 10

⇒ Oui elle s'utilise comme une table dans une requête SELECT

Question 2.4 - A partir de la vue vue_clients, affichez uniquement les clients de l' Etat SP. Combien de lignes sont retournées ?

SELECT * FROM vue_clients WHERE customer_state = 'SP';

⇒ Elle retourne 41746 lignes

Question 2.5 - Quel est l'intérêt des vue_clients par rapport à la table complète customers ?

⇒ L'intérêt de vue_clients par rapport à la table complète customers est qu'elle n'expose que 3 colonnes utiles (customer_id, customer_city, customer_state) au lieu de toutes les colonnes de la table. Cela apporte plusieurs avantages :

- **Simplicité** : l'utilisateur voit uniquement les données dont il a besoin, sans être noyé dans des colonnes inutiles.
- **Sécurité** : les données sensibles comme l'email ou le nom du client restent cachées et inaccessibles via la vue.
- **Lisibilité** : les requêtes écrites sur la vue sont plus courtes et plus faciles à comprendre.

Partie 3 - Vue avec filtrage

Question 3.1 - Créez une vue appelée `vue_clients_sp` contenant uniquement les clients de l'Etat SP.

⇒

```
CREATE VIEW vue_clients_sp AS
SELECT *
FROM customers
WHERE customer_state = 'SP';
```

Question 3.2 - Interrogez la vue `vue_clients_sp` et affichez ses dix premières lignes.

⇒

```
SELECT * FROM vue_clients_sp LIMIT 10;
```

Question 3.3 - Vérifiez que tous les clients de cette vue appartiennent bien à l'Etat SP.

⇒

```
SELECT DISTINCT customer_state FROM vue_client_sp;
```

Question 3.4 - Comparez le résultat obtenu avec une requête directe sur `customers` filtrée sur `customer_state = SP`. Les deux résultats sont-ils équivalents ?

⇒ Les deux requêtes retournent les mêmes résultats car la vue `vue_clients_sp` est basée sur exactement la même condition `WHERE customer_state = 'SP'` appliquée à la table `customers`. La vue est donc bien un raccourci vers cette requête

Question 3.5 - Dans quel cas une vue filtrée peut-elle simplifier le travail d'un utilisateur ?

⇒ *Une vue filtrée peut simplifier le travail d'un utilisateur dans plusieurs cas :*

- **Un analyste** qui travaille toujours sur la même région n'a pas besoin de réécrire le `WHERE customer_state = 'SP'` à chaque requête, il appelle directement `vue_clients_sp`.
- **Un développeur** évite de répéter la même condition de filtre dans plusieurs endroits de son application, ce qui réduit les erreurs.
- **Un utilisateur non technique** peut interroger une vue simple sans connaître la structure complète de la table `customers`.
- **La sécurité** : un utilisateur n'a accès qu'aux données de son périmètre, par exemple uniquement les clients de l'état SP, sans voir les autres.

Partie 4 - Vue avec jointure entre clients et commandes

Question 4.1 - Ecrivez une requête qui affiche order_id, order_status, order_purchase_timestamp, customer_id, customer_city et customer_state en utilisant une jointure entre orders et customers.

⇒

```
SELECT o.order_id, o.order_status, o.order_purchase_timestamp, c.customer_id,
       c.customer_city, c.customer_state
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;
```

Question 4.2 - Transformez cette requête en vue appelée vue_commandes_clients.

⇒

```
CREATE VIEW vue_commandes_clients AS
SELECT o.order_id, o.order_status, o.order_purchase_timestamp, c.customer_id,
       c.customer_city, c.customer_state
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;
```

Question 4.3 - Interrogez la vue vue_commandes_clients et affichez ses dix premières lignes.

⇒

```
SELECT *
FROM vue_commandes_clients
LIMIT 10;
```

Question 4.4 - A partir de cette vue, affichez toutes les commandes passées par des clients de l'Etat RJ.

⇒

```
SELECT *
FROM vue_commandes_clients
WHERE customer_state = 'RJ';
```

Question 4.5 - A partir de cette vue, affichez le nombre de commandes par Etat.

⇒

```
SELECT customer_state, COUNT(*) AS nombre_commande
FROM vue_commandes_clients
GROUP BY customer_state
ORDER BY nombre_commandes DESC;
```

Question 4.6 - Quel Etat possède le plus grand nombre de commandes ?

⇒ L'état qui a le plus grand nombre de commandes est le SP avec 41746

Question 4.7 - Pourquoi cette vue est-elle plus pratique qu'une jointure réécrite à chaque fois ?

⇒ Cette vue est plus pratique car elle évite de réécrire la jointure à chaque fois, simplifie le code et réduit les erreurs.

Partie 5 - Vue avec jointure entre commandes et paiements

Question 5.1 - Ecrivez une requête qui affiche order_id, order_status, order_purchase_timestamp, payment_type, payment_installments et payment_value a partir des tables orders et payments.

⇒
SELECT
o.order_id,o.order_status,o.order_purchase_timestamp,p.payment_type,p.payment_installments,p.payment_value
FROM orders o
JOIN payments p ON o.order_id = p.order_id;

Question 5.2 - Creez une vue appelee vue_commandes_paiements a partir de cette requete.

⇒
CREATE VIEW vue_commandes_paiements AS
SELECT
o.order_id,o.order_status,o.order_purchase_timestamp,p.payment_type,p.payment_installments,p.payment_value
FROM orders o
JOIN payments p ON o.order_id = p.order_id;

Question 5.3 - Affichez les dix premieres lignes de vue_commandes_paiements.

⇒
SELECT *
FROM vue_commandes_paiements
LIMIT 10;

Question 5.4 - A partir de cette vue, calculez le montant total paye par type de paiement.

⇒
SELECT payment_type , SUM(payment_value) AS paye
FROM vue_commandes_paiements
GROUP BY payment_type;

Question 5.5 - A partir de cette vue, calculez le nombre de paiements par type de paiement.

⇒
SELECT payment_type , COUNT(*) AS payé
FROM vue_commandes_paiements
GROUP BY payment_type;

Question 5.6 - Quel type de paiement est le plus fréquent ?

⇒ Le type de paiement le plus fréquent est credit_card avec 153590.

Question 5.7 - Quel type de paiement représente le montant total le plus élevé ?

⇒ Le type de paiement représente le montant total le plus élevé est credit_card avec 153590.

Partie 6 - Vue avec produits et articles commandes

Question 6.1 - Ecrivez une requête qui affiche order_id, product_id, product_category_name, price et freight_value en combinant order_items et products.

⇒

```
SELECT o.order_id, o.product_id, p.product_category_name, o.price, p.freight_value
FROM order_items o
JOIN products p ON o.product_id = p.product_id;
```

Question 6.2 - Créez une vue appelee vue_articles_produits a partir de cette requête.

⇒

```
CREATE VIEW vue_articles_produits AS
SELECT o.order_id, o.product_id, p.product_category_name, o.price, o.freight_value
FROM order_items o
JOIN products p ON o.product_id = p.product_id;
```

Question 6.3 - Interrogez la vue vue_articles_produits et affichez ses dix premieres lignes.

⇒

```
SELECT *
FROM vue_articles_produits
LIMIT 10;
```

Question 6.4 - A partir de cette vue, affichez le chiffre d'affaires par catégorie de produit.

⇒

```
SELECT product_category_name, SUM(price) AS chiffre_affaires
FROM vue_articles_produits
GROUP BY product_category_name
ORDER BY prix_moyen DESC;
```

Question 6.5 - A partir de cette vue, affichez le prix moyen par catégorie de produit.

⇒

```
SELECT product_category_name, AVG(price) AS prix_moyen
FROM vue_articles_produits
GROUP BY product_category_name
ORDER BY prix_moyen DESC;
```

Question 6.6 - Quelle catégorie génère le plus grand chiffre d'affaires ?

⇒ Le catégorie qui génère le plus grand chiffre d'affaires est beleza_saude avec 5034725.36

Question 6.7 - Quelle catégorie possède le prix moyen le plus élevé ?

⇒ Le catégorie qui possède le plus grand chiffre d'affaires est pcs avec 1098.340542

Partie 7 - Vue analytique : ventes détaillées

Question 7.1 - Ecrivez une requête qui combine customers, orders, order_items et products pour afficher order_id, customer_state, product_category_name, price et freight_value.

⇒

```
SELECT o.order_id, c.customer_state, p.product_category_name, oi.price,
oi.freight_value
FROM customers c
```

```
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

Question 7.2 - Creez une vue appelée vue_ventes_detaillees a partir de cette requête.

```
⇒ CREATE VIEW vue_ventes_detaillees AS
SELECT
    o.order_id,
    c.customer_state,
    p.product_category_name,
    oi.price,
    oi.freight_value
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

Question 7.3 - Affichez les dix premières lignes de vue_ventes_detaillees.

```
⇒ SELECT * FROM vue_ventes_detaillees LIMIT 10;
```

Question 7.4 - A partir de cette vue, calculez le chiffre d affaires total par Etat.

```
⇒ SELECT customer_state,SUM(price) AS chiffre_affaire
FROM vue_ventes_detaillees
GROUP BY customer_state;
```

Question 7.5 - A partir de cette vue, calculez le chiffre d affaires par Etat et par categorie de produit.

```
⇒ SELECT customer_state,product_category_name,SUM(price) AS chiffre_affaire
FROM vue_ventes_detaillees
GROUP BY customer_state,product_category_name;
```

Question 7.6 - A partir de cette vue, calculez pour chaque État : le nombre de commandes, le chiffre d' affaires et le panier moyen.

```
⇒ SELECT
    customer_state,
    COUNT(order_id) AS nombre_commandes,
    SUM(price) AS chiffre_affaire,
    AVG(price) AS panier_moyen
FROM vue_ventes_detaillees
GROUP BY customer_state;
```

Question 7.7 - Quel État génère le chiffre d' affaires le plus élevé ?

```
⇒ L'état qui génère le chiffres d'affaires le plus élevé est MT avec 355.79
```

Question 7.8 - Quel État possède le panier moyen le plus élevé ?

```
⇒ L'état qui possède le panier moyen le plus élevé est MT avec 355.790000
```

Partie 8 - Vue avec agrégation

Question 8.1 - Creez une vue appelee vue_ca_par_categorie contenant product_category_name,nombre_articles_vendus, chiffre_affaires et prix_moyen.

```
⇒ CREATE VIEW vue_ca_par_categorie AS
  SELECT p.product_category_name,
         COUNT(oi.order_id) AS nombre_articles_vendus,
         SUM(oi.price) AS chiffre_affaires,
         AVG(oi.price) AS prix_moyen
  FROM order_items oi
  JOIN products p ON oi.product_id = p.product_id
  GROUP BY p.product_category_name;
```

Question 8.2 - Interrogez la vue vue_ca_par_categorie en triant les résultats par chiffre d affaires décroissant.

```
⇒ SELECT *
  FROM vue_ca_par_categorie
 ORDER BY chiffre_affaires DESC;
```

Question 8.3 - Affichez uniquement les catégories ayant un chiffre d affaires superieur a 100 000.

```
⇒ SELECT *
  FROM vue_ca_par_categorie
 WHERE chiffre_affaires > 100000;
```

Question 8.4 - Quels sont les avantages d' une vue contenant déjà une jointure et un GROUP BY ?

L'avantage d'une vue contenant déjà une jointure et un GROUP BY est qu'il n'est plus nécessaire de les réécrire à chaque fois. Cela simplifie les requêtes, réduit les erreurs et permet à n'importe quel utilisateur d'obtenir des résultats agrégés en faisant simplement un SELECT * FROM la_vue.

Question 8.5 - Quelles limites peut avoir une vue avec agrégation si les donnees changent souvent ?

⇒ Une vue avec agrégation peut devenir lente sur de grandes tables car elle recalcule les résultats à chaque appel. De plus, elle ne peut pas être mise à jour directement comme une table classique.

Partie 9 - Vue de reporting

Question 9.1 - Créez une vue appelee vue_reporting_commandes donnant pour chaque Etat et chaque statut de commande : customer_state, order_status et nombre_commandes.

```
⇒ CREATE VIEW vue_reporting_commandes AS
  SELECT
    c.customer_state,
    o.order_status,
```

```
        COUNT(o.order_id) AS nombre_commandes
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_state, o.order_status;
```

Question 9.2 - Interrogez cette vue en triant par Etat puis par nombre de commandes décroissant.

```
⇒ SELECT *
   FROM vue_reporting_commandes
   ORDER BY customer_state, nombre_commandes DESC;
```

Question 9.3 - À partir de cette vue, affichez uniquement les commandes ayant le statut delivered.

```
⇒ SELECT *
   FROM vue_reporting_commandes
   WHERE order_status = 'delivered';
```

Question 9.4 - Quel est l'intérêt d'une vue de reporting pour un responsable commercial ?

⇒ Une vue de reporting permet à un responsable commercial de consulter rapidement ses indicateurs clés sans connaître le SQL. Elle lui évite de dépendre d'un développeur pour chaque demande et lui fournit des données claires et prêtes à l'emploi.

Question 9.5 - Comment cette vue pourrait-elle être utilisée dans un tableau de bord ?

⇒ Cette vue pourrait être connectée directement à un outil de tableau de bord comme Power BI ou Tableau. Ces outils interrogent la vue automatiquement et affichent les résultats sous forme de graphiques et de tableaux, sans que l'utilisateur ait besoin d'écrire une seule ligne de SQL.

Partie 10 - Vue et securite des donnees

Question 10.1 - Creez une vue appelee vue_clients_public qui ne montre que customer_city et customer_state.

```
⇒ CREATE VIEW vue_clients_public AS
   SELECT customer_city, customer_state
   FROM customers;
```

Question 10.2 - Interrogez la vue vue_clients_public et affichez ses dix premieres lignes.

```
⇒ SELECT * FROM vue_clients_public LIMIT 10;
```

Question 10.3 - Pourquoi cette vue peut-elle être utile si on veut donner un accès limité à certains utilisateurs ?

⇒ Cette vue est utile car elle ne montre que les données non sensibles comme la ville et l'état, sans exposer des informations personnelles comme le nom ou l'email du client. On peut donc donner accès à cette vue à certains utilisateurs sans leur donner accès à toute la table customers

Question 10.4 - Quelles colonnes de la table customers sont volontairement masquées dans cette vue ?

⇒ Les colonnes masquées sont `customer_id`, `customer_unique_id`, `customer_zip_code_prefix` et toute autre information personnelle comme le nom ou l'email. Seules `customer_city` et `customer_state` sont visibles, car ce sont des données non sensibles.

Question 10.5 - Une vue suffit-elle toujours à garantir la sécurité des données ? Justifiez brièvement.

⇒ Non, une vue ne suffit pas toujours à garantir la sécurité des données. Si un utilisateur a accès direct à la table en plus de la vue, il peut contourner les restrictions. Il faut donc combiner les vues avec une gestion des droits d'accès en accordant uniquement les permissions nécessaires à chaque utilisateur.

Partie 11 - Modification et suppression d'une vue

Question 11.1 - Supprimez la vue `vue_clients_public`.

⇒ `DROP VIEW vue_clients_public;`

Question 11.2 - Que se passe-t-il si vous essayez ensuite d'interroger `vue_clients_public` ?

⇒ Si on essaie d'interroger `vue_clients_public` après l'avoir supprimée, MySQL retournera une erreur indiquant que la vue n'existe pas :
ERROR 1146: Table 'vue_clients_public' doesn't exist.
La vue ayant été supprimée avec `DROP VIEW`, elle n'est plus disponible.

Question 11.3 - Recréez `vue_clients_public` pour afficher `customer_state`, `customer_city` et nombre_clients par ville et par État.

⇒ `CREATE VIEW vue_clients_public AS`
`SELECT customer_state, customer_city, COUNT(customer_id) AS nombre_clients`
`FROM customers`
`GROUP BY customer_state, customer_city;`

Question 11.4 - Interrogez la nouvelle version de la vue en triant par nombre_clients décroissant.

⇒ `SELECT *`
`FROM vue_clients_public`
`ORDER BY nombre_clients DESC;`

Question 11.5 - Quelle ville contient le plus de clients dans le dataset ?

⇒ La ville avec le plus de clients est São Paulo, car elle apparaît en première position après le tri par nombre_clients décroissant.

Question 11.6 - Quelle est la différence entre supprimer une vue et supprimer une table ?

⇒ La différence est que supprimer une table avec `DROP TABLE` supprime définitivement la table et toutes ses données physiques. En revanche, supprimer une vue avec `DROP VIEW` ne supprime que la définition de la requête, sans affecter les données de la table source.

Partie 12 - Vue et performance

Question 12.1 - Exécutez directement une requête calculant le chiffre d'affaires par catégorie sans utiliser de vue. Notez le temps d'exécution affiché par votre outil.

⇒

```
SELECT p.product_category_name, SUM(oi.price) AS chiffre_affaires
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY chiffre_affaires DESC;
```


74 rows in set (2,61 sec)

Question 12.2 - Exécutez la même analyse à partir de vue_articles_produits. Notez le temps d'exécution.

⇒

```
SELECT product_category_name, SUM(price) AS chiffre_affaires
FROM vue_articles_produits
GROUP BY product_category_name
ORDER BY chiffre_affaires DESC;
```


74 rows in set (2,54 sec)

Question 12.3 - Le temps d'exécution est-il très différent entre la requête directe et la requête via la vue ?

⇒ Non le temps d'exécution n'est pas très différent il n'y a que 0,07 sec de différence

Question 12.4 - Une vue classique améliore-t-elle toujours les performances ? Justifiez.

⇒ Non, une vue classique n'améliore pas toujours les performances. Elle exécute la même requête à chaque appel, donc le temps d'exécution est similaire à la requête directe comme on vient de le constater.

Question 12.5 - A quoi sert principalement une vue classique si elle n'accélère pas toujours les requêtes ?

⇒ Une vue classique sert principalement à ne pas réécrire la requête à chaque fois, à simplifier le code et à sécuriser l'accès aux données.

Question 12.6 - Quels index pourraient aider les requêtes utilisées par les vues précédentes ?

⇒ Les index qui pourraient aider sont :

- `product_id` dans `order_items` et `products` pour accélérer les jointures
- `order_id` dans `orders` et `order_items` pour accélérer les jointures
- `customer_id` dans `customers` et `orders` pour accélérer les jointures

Partie 13 -Vue matérialisée ou table de synthèse

Question 13.1 - Expliquez la différence entre une vue classique et une vue matérialisée.

⇒ La différence entre une vue classique et une vie matérialisée est qu'une vue classique est une requête virtuelle qui recalcule les données en temps réel, tandis qu'une vue matérialisée stocke physiquement le résultat d'une requête sur le disque (comme une table) et doit être actualisée.

Question 13.2 - Sous PostgreSQL, créez une vue matérialisée donnant le chiffre d'affaires par catégorie.

⇒

```
CREATE MATERIALIZED VIEW vue_ca_par_categorie_mat AS
SELECT p.product_category_name,
       SUM(oi.price) AS chiffre_affaires
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY chiffre_affaires DESC;
```

Question 13.3 - Interrogez la vue matérialisée et comparez son temps d'exécution avec la vue classique.

⇒

```
SELECT * FROM vue_ca_par_categorie_mat;
```

Question 13.4 - Pourquoi faut-il rafraîchir une vue matérialisée lorsque les données sources changent ?

⇒ Il faut rafraîchir une vue matérialisée car elle stocke physiquement les données au moment de sa création.

Question 13.5 - Sous MySQL, créez une table de synthèse qui simule une vue matérialisée pour le chiffre d'affaires par catégorie.

⇒

```
CREATE TABLE synthese_ca_par_categorie AS
SELECT p.product_category_name, SUM(oi.price) AS chiffre_affaires
FROM order_items oi
JOIN products p
ON oi.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY chiffre_affaires DESC;
```

Question 13.6 - Comparez la table de synthèse MySQL avec une vue classique : avantages, limites et risques.

	Vue classique	Table de synthèse MySQL
Avantages	Toujours à jour automatiquement, ne prend pas d'espace disque	Rapide à interroger car données stockées physiquement
Limites	Lente sur de grandes tables car recalcule à chaque appel	Nécessite une mise à jour manuelle quand les données changent

Risques	Performances dégradées sur des requêtes complexes	Données obsolètes si on oublie de la mettre à jour
---------	---	--

Partie 14 -Exercices de synthèse

Question 14.1 - Creez une vue appelee vue_commandes_livrees contenant uniquement les commandes dont le statut est delivered.

```
⇒ CREATE VIEW vue_commandes_livrees AS
    FROM orders
    WHERE order_statut = 'delivered';
```

Question 14.2 - A partir de vue_commandes_livrees, affichez le nombre de commandes livrees par mois.

```
⇒ SELECT MONTH(order_purchase_timestamp) AS mois,
    COUNT(*) AS nombre_commandes
    FROM vue_commandes_livrees
    GROUP BY MONTH(order_purchase_timestamp)
    ORDER BY mois;
```

Question 14.3 - Creez une vue appelée vue_retard_livraison permettant d'afficher les commandes livrées en retard.

```
⇒ CREATE VIEW vue_retard_livraison AS
    SELECT order_id,
    customer_id,
    order_status,
    order_estimated_delivery_date,
    order_delivered_customer_date,
    DATEDIFF(order_delivered_customer_date, order_estimated_delivery_date) AS
    jours_retard
    FROM orders
    WHERE order_status = 'delivered'
    AND order_delivered_customer_date > order_estimated_delivery_date;
```

Question 14.4 - A partir de vue_retard_livraison, calculez le nombre total de commandes en retard.

```
⇒ SELECT COUNT(*) AS nombre_total
    FROM vue_retard_livraison;
```

Question 14.5 - Créez une vue appelée vue_top_categories donnant le chiffre d'affaires, le nombre d'articles vendus et le prix moyen par catégorie.

```
⇒ CREATE VIEW vue_top_categories AS
    SELECT p.product_category_name,
    SUM(oi.price) AS chiffre_affaires,
    COUNT(oi.order_id) AS nombre_articles_vendus,
    AVG(oi.price) AS prix_moyen
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
```



```
GROUP BY p.product_category_name  
ORDER BY chiffre_affaires DESC;
```

Question 14.6 - A partir de vue_top_categories, affichez les dix categories les plus rentables.

```
⇒ SELECT *  
FROM vue_top_categories  
ORDER BY chiffre_affaires DESC  
LIMIT 10;
```

Question 14.7 - Créez une vue appelée vue_paiements_resume donnant, pour chaque type de paiement, le nombre de paiements, le montant total et le montant moyen.

```
⇒ CREATE VIEW vue_paiements_resume AS  
SELECT payment_type,  
       COUNT(*) AS nombre_paiements,  
       SUM(payment_value) AS montant_total,  
       AVG(payment_value) AS montant_moyen  
FROM payments  
GROUP BY payment_type;
```

Question 14.8 - A partir de vue_paiements_resume, identifiez le type de paiement qui génère le montant total le plus eleve.

```
⇒ SELECT *  
FROM vue_paiements_resume  
ORDER BY montant_total DESC  
LIMIT 1;
```

Question 14.9 - Créez une vue de votre choix permettant de répondre à une question métier pertinente sur la base Olist.

```
⇒ CREATE VIEW vue_performance_vendeurs AS  
SELECT oi.seller_id,  
       COUNT(oi.order_id) AS nombre_commandes,  
       SUM(oi.price) AS chiffre_affaires,  
       AVG(oi.price) AS prix_moyen,  
       SUM(oi.freight_value) AS total_livraison  
FROM order_items oi  
GROUP BY oi.seller_id  
ORDER BY chiffre_affaires DESC;  
  
SELECT *  
FROM vue_performance_vendeurs  
LIMIT 10;
```

Question 14.10 - Expliquez en quelques lignes l'utilité de la vue que vous avez créée a la question précédente.

⇒ La vue vue_performance_vendeurs permet d'identifier rapidement les vendeurs les plus performants sur la plateforme Olist. Elle est utile pour un responsable commercial qui

souhaite récompenser les meilleurs vendeurs ou identifier ceux qui ont besoin d'être accompagnés. Grâce à cette vue, il peut prendre des décisions sans écrire de requête complexe, en consultant simplement le chiffre d'affaires, le nombre de commandes et le prix moyen de chaque vendeur.

Partie 15 - Questions de réflexion

Question 15.1 - Dans quels cas une vue est-elle utile dans un projet réel ?

⇒ Une vue est utile pour simplifier des requêtes complexes, sécuriser l'accès aux données et réutiliser une requête sans la réécrire.

Question 15.2 - Pourquoi une vue peut-elle améliorer la lisibilité du code SQL ?

⇒ Une vue remplace une longue requête par un simple nom, ce qui rend le code plus court et plus facile à comprendre.

Question 15.3 - Pourquoi une vue classique n'améliore-t-elle pas toujours les performances ?

⇒ Une vue classique n'améliore pas toujours les performances car elle ne stocke pas les résultats physiquement. Elle réexécute la requête à chaque appel, donc plus la requête est complexe avec des jointures et des agrégations, plus elle sera lente.

Question 15.4 - Quelle est la différence entre une vue simple et une vue avec agrégation ?

⇒ La différence entre une vue simple et une vue avec agrégation est que la vue simple affiche des données brutes, tandis que la vue avec agrégation calcule des résultats groupés comme des sommes ou des moyennes.

Question 15.5 - Pourquoi faut-il parfois utiliser une vue matérialisée ou une table de synthèse au lieu d'une vue classique ?

⇒ Il faut parfois utiliser une vue matérialisée ou une table de synthèse car elles stockent physiquement les résultats, ce qui les rend beaucoup plus rapides à interroger sur de grandes tables.

Question 15.6 - Quels sont les risques d'utiliser trop de vues imbriquées les unes dans les autres ?

⇒ Les vues imbriquées peuvent rendre le code difficile à maintenir, ralentir les requêtes et compliquer le débogage.

Question 15.7 - Une vue peut-elle être utilisée pour masquer certaines colonnes sensibles ?

⇒ Oui, une vue peut n'exposer que certaines colonnes et cacher les données sensibles comme les emails ou les salaires.

Question 15.8 - Dans un projet décisionnel ou de reporting, pourquoi les vues sont-elles importantes ?

⇒ Les vues centralisent les indicateurs clés et permettent aux analystes d'accéder aux données sans connaître le SQL.

Question 15.9 - Quelle différence faites-vous entre simplification logique et optimisation physique ?

⇒ La simplification logique facilite la lecture du code, tandis que l'optimisation physique améliore réellement les performances d'exécution.

Question 15.10 - Faut-il remplacer toutes les requêtes complexes par des vues ? Justifiez.

⇒ Non, il ne faut pas remplacer toutes les requêtes complexes par des vues. Une vue est utile quand une requête est réutilisée souvent, mais si elle n'est utilisée qu'une seule fois, créer une vue est inutile et alourdit la base de données. Il faut donc créer une vue uniquement quand elle apporte un réel avantage en termes de simplicité, sécurité ou réutilisabilité.